

Navn: Allan Sigge Bruun Andersen	2018.4
Fag: Programmering B	Vejleder: Mette Frost Nielsen E-mail: mfn@rts.dk
Fag: Matematik A	Vejleder: Mette Frost Nielsen E-mail: mfn@rts.dk

Emne/område:	Kryptering og RSA
Opgaveformulering: Beskriv hvad kryptering er og forklar forskellen på symmetrisk og asymmetrisk kryptering. Gør rede for hvad primtal og regning med rester er og kom ind på væsentlige sammenhænge der gælder inden for de to begreber. Beskriv algoritmen bag RSA kryptering, og forklar hvordan den virker, kom især ind på hvordan primtal og regning med rester indgår. Vis et eksempel med udregning af nøgler, når de to første primtal vælges med værdierne $p=419$ og $q=809$. Udarbejd et program som kan kryptere og dekryptere ved hjælp af udvalgte pointer fra RSA kryptering. Diskuter hvilken indflydelse kryptering har på vores teknologier i dag og vurder hvordan den fortsatte teknologiske udvikling vil påvirke de krypteringsalgoritmer, der anvendes i dag. Besvarelsen skal have et omfang på 15-20 normalsider á 2.400 anslag inkl. mellemrum og ekskl. forside, indholdsfortegnelse og bilag. Besvarelsen skal også indeholde et resume på 10-20 linier. Allan Sigge Bruun Andersen har, i forbindelse med afleveringen på www.netprøver.dk, bekræftet på tro og love, ikke at have snydt og selv at have udfærdiget besvarelsen.	

Kryptering - RSA

//Studie Område Projekt

Af: Allan Sigge Bruun Andersen

Vejleder: Mette Frost Nielsen

Fag: Mat A & Pro B

Skole: HTX Roskilde

Klasse: 2018.4 / 3.4

Aflevering: 18/12-2020 Kl. 12.00

Tegn: 44.884

Resume

Jeg vil i denne opgave introducere til emnet kryptering, med fokus på asymmetrisk kryptering. Jeg vil derudover gennemregne krypteringseksempler vha. formler jeg udleder. De matematiske formler udleder jeg vha. givne sætninger som er bevist. Jeg vil ikke fokusere på matematikken bag sætningerne, men bruge sætningerne fremadrettet. Sætningerne jeg tager udgangspunkt i er Eulers sætning, og i samspil med Euklids algoritme har man en del af fundamentet for RSA kryptering. Metoderne vil blive brugt sammen med modulo funktionen da meget af sikkerheden i RSA også ligger i princippet om ikke at udgive mere information end hvad er absolut nødvendigt. Fremtiden for kryptering bliver påvirket meget af den teknologiske udvikling da teknikken bliver bedre og hurtigere til at lave de beregninger der eventuelt ville kunne bryde koden for en asymmetrisk kryptering. Jeg diskuterer herefter fordele/ulemper og hvordan man kan effektivisere processen.

Indholdsfortegnelse

Resume	2
Indledning	4
Redegørelse af metoder	4
Primtal	5
Indbyrdes primiske tal	6
Rester & restklasser	7
Euklids algoritme	8
RSA.....	10
Matematikken bag.....	11
Overordnede formler.....	11
Eksempel.....	12
Programmet	14
Krav.....	14
Omskrivning, fra matematik til algoritme.....	15
Euklids algoritme	15
Faktorisering.....	17
Programmet & kode	18
Test	21
Diskussion/Vurdering	22
Perspektivering	23
Konklusion.....	24
Litteraturliste	25
Bilag	26

Indledning

Hvordan holder man informationer sikre på nettet, hvor alle kan se hvad der bliver sendt? Kryptering har i mange år holdt information der skal sendes over nettet sikre fra at hackere kan få fat i dem. Vores informationer er heldigvis sikret af smarte matematiske formler, der bestyrker vores data. Men hvornår er vores data ikke sikker længere? I denne opgave vil jeg prøve at finde svar på nogle af de spørgsmål der kan stilles ved sikkerhed på nettet, lave en forklaring af krypteringsmetoder og redegøre for hvorfor du kan føle dig sikker på nettet.

Kryptering er ikke et nyt og banebrydende emne, men det er alligevel interessant hvordan man kan lave information der kun er brugbart ved at kende nogle helt bestemte værdier, og hvordan man kan bruge 2 nøgler til en kryptering. Lige netop disse spørgsmål stiller jeg, og jeg vil svare på dem ved at udlede en sammenhæng af matematikken bag kryptering og ydermere udarbejde et program der kan lave en kryptering der i princippet virker ligesom den der sker når vi begår os på nettet.

Redegørelse af metoder

Kryptering er et kendt emne, og der findes utallige metoder til at kryptere information, beskeder og meget andet som er blevet brugt i mange år. Generelt set kan man inddеле alle metoder i to grupper, nemlig symmetriske og asymmetriske krypteringer.

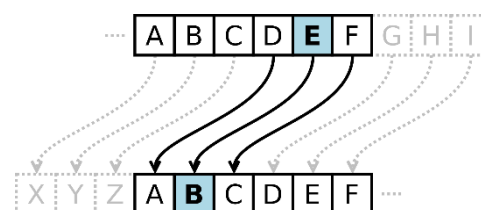
I en symmetrisk kryptering vil man typisk ændre på bogstaverne, ligesom en af de første koder der blev opfundet, Cæsar koden. I Cæsar koden forskyder man i bund og grund alfabetet. Hvis vi antog at A = 1, B = 2, C = 3 osv. Kunne man altså lægge eks. 3 til, for at kryptere beskeden. På den måde giver ordene ikke mening for mennesker og for mange år siden var den besværlig, dermed sikker nok til at Julius Cæsar brugte denne.¹ Metoden er illustreret på Figur 1.

I dag bruges den ikke så ofte da man kan knække en sådan kode ved frekvensanalyse, hvor man analyserer hyppigheden af alle bogstaver og dermed kan bestemme nøglen.

For at dekryptere en sådan besked vil man altså trække det bestemte antal pladser fra. For at kryptere laver man altså en proces, og for at dekryptere laver man den inverse proces, eller man kan sige at man bruger den samme nøgle ved begge processer.

I dag kendes denne også som A-K koden og bruges bl.a. af spejdere.

Ved en asymmetrisk kryptering bruges der forskellige nøgler ved kryptering og dekryptering. Der findes altså offentlige og private nøgler. Ens offentlige nøgle skal altså bruges af alle der vil sende en noget, og kan offentliggøres i noget man kunne sammenligne med en telefonbog. Der sker altså en form for envejsproces idet man krypterer indholdet, og her er det bemærkelsesværdigt at man selv med den offentlige nøgle ikke kan komme tilbage igen når først beskeden er krypteret.



Figur 1 Kilde: Wikipedia

¹ Wikipedia, "Caesar cipher", Wikipedia.org. Besøgt 9/12 2020

Når beskeden skal dekrypteres, sker der altså en anden envejsproces for at man kunne læse beskeden. På denne måde sikrer man sig altså bedre fra at andre kan få adgang til den information der sendes over nettet. Jeg vil undersøge nærmere hvordan disse envejsprocessor virker, og hvor sikkerheden i dette system ligger.

For at sammenligne de to metoder kan man forestille sig en postkasse. Ved en symmetrisk kryptering har postkassen kun en åbning, og alle der skal ind i postkassen skal bruge samme nøgle, uanset om man vil ligge et brev eller tømme postkassen. Nøglen er altså den samme, men metoden er den omvendte. Ved en asymmetrisk kryptering har postkassen 2 åbninger, til hver åbning skal der bruges en nøgle, altså næsten som vi kender en postkasse. Vi ved at man skal bruge en nøgle til at åbne postkassen, men for at lægge post i skal man også bruge en nøgle. Denne kalder man den offentlige nøgle. Her deler man altså op i hvem der skal aflevere post og hvem der skal modtage den. Her har ejeren den eneste nøgle der kan tømme postkassen.

Hvis man vil bruge kryptering i forbindelse med computere, findes der selvfølgelig også vanskeligheder ved metoderne. Fordelen ved en klassisk symmetrisk kryptering er at den er ret nem og hurtig at lave. Derimod er den ikke ret sikker; Når 2 enheder skal udveksle informationer skal de "aftale" hvilken nøgle de bruger, så informationerne bliver brugbare. Her har man problematikken med at man skal udveksle nøglen ved brug af en symmetrisk kryptering, for før man har aftalt en sikker nøgle, skal nøglen sendes. Det vil altså sige at alle der lytter med på linjen får nøglen alligevel, og al information der bliver sendt som bruger denne nøgle kan dekrypteres af alle der "lytter" med. Hvis man derimod bruger en asymmetrisk kryptering, her denne en væsentlig større fordel. Idet man i metoden bruger 2 forskellige nøgler, behøver man ikke at beskytte den offentlige nøgle som er den eneste man skal sende ud for at kommunikere. Her er den asymmetriske metode en meget stærkere fundamentalt da man udelukkende skal udveksle offentlige nøgler som ikke er følsomme informationer ligesom den symmetriske metode.

Jeg vil senere gå mere i dybden omkring hvordan en asymmetrisk metode, RSA, virker og senere diskutere hvordan og hvorfor man kan lave denne slags kryptering.

Primtal

Primtal er tal hvor ingen tal, på nær 1 og tallet selv, går op. Primtal er en vigtig del af RSA algoritmen, hvorfor vil jeg komme ind på senere, men i store træk er primtal stærke at bruge i kryptering fordi et tal bestående af to primtal multiplicerede er svært at faktorerer, dette kaldes primtalsfaktorisering.

Når vi ønsker at beskytte vores data, kan man altså udnytte netop konceptet med at det er tidskrævende at bestemme disse faktorer.

I praksis tager man udgangspunkt i to store primtal. Når man siger store kan disse være op mod 600 cifre lange, men i denne rapport vil jeg holde tallene så simple at man kan forholde sig til, og regne med tallene. De to primtal kaldes p og q , og nu vil jeg lave et tredje tal, n , således:

$$p \cdot q = n$$

I RSA findes der offentlige og private nøgler. De offentlige skal bruges af alle der vil sende en besked, og de private skal altså kun bruges af en selv og er hemmelige! Tallet n er altså en offentlig nøgle, hvor det er en uoverskuelig opgave at skulle finde p og q . Når man regner med primtal er der altså ingen divisorer til p og q og derfor kun 2 divisorer til n . Til tallet n er der ingen andre divisorer

end netop de 2 tal. Når man så har primtal på alt fra 100 til 617 cifre² og der kun er de 2 divisorer, bliver opgaven om at skulle bestemme p og q uoverskuelige.

Når to primtal multipliceres, er det umuligt effektivt at faktorisere store tal. Her hviler hele sikkerheden på at uvedkommende ikke får fat i den hemmelige del af dataoverførslen.

Grunden til at man skal beskytte primtallene p og q , er fordi de skal bruges til at dekryptere beskeden. Først gennemgår de Eulers ϕ funktion:

$$\phi(pq) = (p - 1) \cdot (q - 1) \quad \text{Sætning 1}$$

Herefter skal man bruge resultatet til Euklids algoritme som jeg vil forklare i afsnittet om matematikken bag.

Indbyrdes primiske tal

Endnu et vigtigt udtryk som jeg skal bruge er som overskriften siger, indbyrdes primiske tal.

Man siger, at to tal a og b er indbyrdes primiske hvis de eneste tal der går op i begge tal er 1 eller -1. Det vil altså sige, at 3 og 10 er indbyrdes primiske, selvom 5 går op i 10. Det afhænger altså ikke om hvorvidt tallene er primtal, men blot om der er et tal der går op i begge tal. 13 og 52 er altså ikke indbyrdes primiske da 13 går op i både 13 og 52.

Hvis man vil være lidt smart og gøre det nemmere for en selv når man skal vælge 2 indbyrdes primiske tal, p og q , kan man vælge det mindste tal(p) som et primtal, for da skal man ikke tænke på om der er nogle tal der går op i p , og derefter skal man kun tjekke efter om p går op i q (det større tal). Det er klart, hvis p er et primtal, er der ingen tal der går op i dette, og da alle tal større end p ikke kan gå op i p i hele tal, skal man udelukkende sørge for at q ikke kan divideres med p .

Antallet af tal der er indbyrdes primiske med tallet p , kaldes $\phi(p)$. Hvis tallet p er et primtal, gælder det at $\phi(p) = p - 1$.

For 2 primtal, gælder: $\phi(p) \cdot \phi(q) = (p - 1) \cdot (q - 1) = \phi(pq)$

Og herfra er det bestemt at hvis a og b er indbyrdes primiske, eksisterer der hele tal u og v således at:

$$a \cdot u + b \cdot v = 1$$

Derfor må det også kunne gælde at:

$$a \cdot u - b \cdot v = 1$$

Da man bare ændrer et fortegn. Dog er dette en stor forskel når jeg skal bruge sætningen til at dekryptere med.

² Wikipedia, "RSA numbers", Wikipedia.org, RSA versioners antal cifre i nøgler. Besøgt 9/12 2020

Rester & restklasser

Rester, og regning med rester er en essentiel del af RSA algoritmen, og derfor er vi nødt til at forstå hvordan man regner med rester, og hvad restklasser egentlig er.

Vi ved alle at når man laver et divisionstykke er det ikke altid at man får et helt tal, altså er der en rest. Det kunne eksempelvis være;

$$\frac{17}{5} = 3,4 = 3\frac{2}{5} = 3 \text{ med resten } 2$$

Resten kan man skrive eller sige på mange måder, men når vi vil regne med resten, er det altid heltallet som er tilbage man interesserer sig for, altså vil det rigtige svar i ovenstående divisionstykke være at resten er 2. Dette er der lavet en regneproces til at bestemme, modulo. Ved at bruge modulo vil man altså bestemme resten af et divisionsstykke, og herved får man kun resten som svar, altså er divisoren ligegyldig. Et eksempel på et regnestykke med modulo:

$$17 \% 5 = 2$$

Nogle skriver ”mod” i stedet for ”%” men her i opgaven bruger jeg procenttegnet. Når man har med rester at gøre, findes der altså også restklasser. For at have en restklasse skal man altså have en bestemt værdi man bruger modulo funktionen med, i eksemplet bruger jeg 5. For at opskrive restklasser bruges den samme værdi altså hele tiden, og resultatet giver en restklasse hvor andre repræsentanter også kan være. Fra eksemplet er restklassen 2 i spil, med repræsentanten 17. Et andet eksempel med samme restklasse ville altså være:

$$32 \% 5 = 2$$

Da det altså giver den samme restklasse. En ting der er værd at lægge mærke til, er at der findes ligeså mange restklasser som det tal man bruger modulus funktionen til (her 5, man kalder også denne for m).

Når man arbejder indenfor den samme restklasse følger der nogle regler³ som jeg kan udnytte når jeg skal udarbejde mit program:

$$[x]\%m + [y]\%m = [x + y]\%m$$

$$[x]\%m \cdot [y]\%m = [x \cdot y]\%m$$

$$[x^y]\%m = [x]^y\%m$$

³ Baktoft, Allan, *Matematik I virkeligheden*, Forlag Natskyggen 3. Udgave, side 172, eksempel, sætning og bevis

Euklids algoritme

I RSA algoritmen, skal jeg bruge en værdi, u , til når vi skal dekryptere. Til hvad og hvorfor opsamler jeg på i afsnittet om matematikken. Euklids algoritme går i bund og grund ud på at finde den største fælles divisor for 2 tal. Euklid fandt altså på en algoritme, hvor man på et tidspunkt vil få 0, men resten før man får 0 er netop den første fælles divisor for de to valgte tal. En af de tre matematikere der opfandt RSA krypteringen fandt formentlig ud af at man kan bruge Euklids algoritme til at bestemme u og v fra Eulers sætning:

Hvis a og b er indbyrdes primiske, eksisterer der hele tal u og v således at:

$$a \cdot u - b \cdot v = 1 \quad \text{Sætning 2}$$

Her ville man altså have værdier for a og b , og hvis man bruger værdierne for a og b i Euklids algoritme vil man kunne udlede værdierne på denne måde. Hvis man tænker over det er det lidt fantastisk at man kan løse en ligning med 2 ubekendte. Jeg vil prøve at gennemgå måden som RSA arbejder med Euklids algoritme på, med nogle værdier jeg kan forholde mig til, og samtidig prøve at vise princippet bag.

Undervejs i Euklids algoritme vil jeg finde divisorer og rester ud fra to tal, og det fortsætter man med indtil resten er lig én. Valgt 2 tal, a og b , vil det altså sige at jeg i praksis regner ud hvor mange b går op i a , hvortil der er en rest. Næste "omgang" er det b divideret med resten fra første omgang, osv.

Hvis jeg opskriver et eksempel med: $a = 61$ $b = 27$:

$$61 = 27 \cdot 2 + 7$$

$$27 = 7 \cdot 3 + 6$$

$$7 = 6 \cdot 1 + 1$$

Og efter 3 omgange har jeg fundet resten 1.

I første omgang skal man bestemme det højeste antal gange 27 går op i 61 og derefter finde resten. Samme teknik går for de næste omgange. Når jeg har bestemt divisorerne og resterne, kan jeg begynde at lave et udtryk hvor jeg vil bestemme u og v . Jeg vil altså nu starte fra slut, og isolere for resten 1.

$$1 = 7 - 6 \cdot 1$$

Nu vil jeg så indsætte udtrykket for 6, altså 2. omgang isoleret for 6:

$$1 = 7 - (27 - 7 \cdot 3) \cdot 1$$

$$1 = 7 - 27 \cdot 1 + 7 \cdot 3 \cdot 1 = 7 \cdot 4 - 27 \cdot 1$$

Og nu vil jeg indsætte udtrykket for 7, altså 1. omgang isoleret for 7:

$$1 = (61 - 27 \cdot 2) \cdot 4 - 27 \cdot 1$$

$$1 = 61 \cdot 4 - 27 \cdot 8 - 27 \cdot 1$$

Og når man lige samlet udtrykket: $1 = 61 \cdot 4 - 27 \cdot 9$

Nu kan man se at udtrykket er opstillet på samme måde som det fra Eulers sætning:

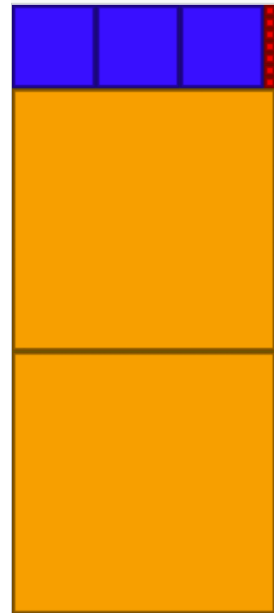
$$1 = a \cdot u - b \cdot v$$

Nu hvor formen er den samme, a og b er de samme værdier, derfor må henholdsvis 4 og 9 være værdierne u og v hvor a = 61 og b = 27.

Euklids algoritme kan også visualiseres således at den giver mere logisk mening. På Figur 2 kan man se tankegangen bag.

På dette stykke jord skal der lægges kvadratiske fliser, de skal være så store som mulige, og derefter skal man udfylde det resterende hul med mindre kvadratiske fliser. For at overføre eksemplet til figuren, er højden af retanklet det største tal, altså 61, og bredden er det andet, 27. Der kan altså ligge 2 fliser der er 27 x 27. Divisoren fra beregningen er altså antallet af fliser der kan ligge. Resten er bredden på de næststørste fliser, hvor der kan ligge D2 (divisoren fra 2. omgang) fliser, altså 3 styk osv.

Senere vil jeg bruge divisorerne (D1, D2 & D3) og resterne (R1, R2 & R3).



Figur 1 Kilde: Wikipedia, Euklids algoritme

RSA

RSA kryptering bygger i bund og grund på matematik og matematiske formler, hvor tal kan indeholde information. Derfor er denne metode også sikker fordi man ikke kan lave en bagdør bare ved at være snu, for så skal man først lave om på matematikkens regler. Fordi man bruger matematik, følger man altså også nogle regneregler som man skal have til at passe ind i den algoritme som man jeg lave.

Selve matematikken er nemlig ret smart, man laver nemlig en envejsfunktion, hvilket betyder at man ikke kan gå tilbage igen, og på den måde bestemmer man hvor meget information man vil sende ud hvor alle kan kigge på det.

Jeg skal huske at indarbejde, og arbejde med de emner og tilhørende regler som jeg har gennemgået. Jeg vil først lave en overordnet gennemgang af hvad jeg skal gøre, og hvor jeg skal bruge hvad.

- Valg af variabler. Her skal man være opmærksom på at k og $\phi(pq)$ er indbyrdes primiske!
- Indtast af besked.
- Kryptering af besked. Her skal jeg bruge faktorisering og bruge modulus n og være opmærksom på restklasserne.
- Dekryptering af besked. Her skal jeg igen bruge faktorisering, modulo funktionen og igen være opmærksom på restklasserne. Udover det skal jeg bruge Euklids algoritme på en måde så jeg kan bruge den i mit program.

Jeg vil tage udgangspunkt i brugerinputs med tal, altså tager jeg ikke højde for hvordan eller med hvilken metode bogstaver eller andre tegn bliver omsat til tal som man krypterer. Senere kan jeg eventuelt implementere en funktion der omsætter fra tekst til tal.

Matematikken bag

Matematikken bag RSA kryptering kan koges rigtig meget ned, men hvis man gør det får man ikke det store billede med. Jeg vil derfor starte med at give den korteste gennemgang af algoritmen, eller rettere matematikken bag.

Jeg vil nu introducere en modtager og en afsender. Derudover vil jeg introducere nogle variabler; P, Q, N, K, M, C og D.

P og Q har jeg nævnt lidt tidligere. P og Q skal være enorme primtal, alt fra 330 til 2048 bit, altså 100-617 cifre lange primtal. (længden afhænger af hvilken version der benyttes) I mine eksempler vil jeg dog holde tallene så små at jeg kan forholde os til dem.

N er produktet af P og Q. K er et tal der skal være indbyrdes primisk med $\phi(pq) = (p - 1) \cdot (q - 1)$.

M er de beskeden, altså her cifre, der skal krypteres. C er ciferteksten, altså den krypterede besked.

D er min variabel for den dekrypterede besked, og skal altså være lig med m.

Nu da vi har det på plads, vil jeg bestemme variablernes værdier:

$$\begin{aligned}p &= 11 \\q &= 17 \\n &= 11 \cdot 17 = 187 \\k &= 7\end{aligned}$$

Jeg kan tillade mig at vælge $k = 7$, da 7 er et primtal og:

$$\begin{aligned}\phi(pq) &= (11 - 1) \cdot (17 - 1) = 160 \\ \frac{160}{7} &\approx 22,857\end{aligned}$$

$$m = 80$$

Overordnede formler

$$c = m^k \% n \quad \text{Formel 1}$$

$$d = c^u \% n \quad \text{Formel 2}$$

Eksempel

Udregning med udvalgte p og q værdier

Ved brug af de værdier jeg valgte før, kan vi altså nemt beregne værdien af c:

$$c = 80^7 \% 187 = 75$$

Udregning 1

Som man kan se, får jeg 75 og så simpelt er beregningen i princippet. Jeg vil nu lave en grundigere gennemgang med udregninger, men da vil jeg bruge lidt højere værdier.

$$p = 419$$

$$q = 809$$

$$n = 419 \cdot 809 = 338.971$$

$$k = 7$$

$$\phi(pq) = (419 - 1) \cdot (809 - 1) = 337.744$$
$$\left(\frac{337.744}{7} \approx 48.249,14 \right)$$

$$m = 80$$

Jeg vil nu, ligesom før, kryptere min besked som ikke er så kompliceret:

$$c = m^k \% n$$

$$c = 80^7 \% 338.971 = 140.307$$

Som formelen for den dekrypterede besked viser, skal jeg først bestemme u:

$$d = c^u \% n$$

Jeg bruger altså Eulers sætning, og Euklids algoritme. Her bliver tingene lidt langhårede når man laver beregningerne i hånden, men husk at jeg bare skal finde u og så har jeg den dekrypterede besked.

Den generelle formel, hvor a og b kan defineres ud fra de værdier jeg kender:

$$a = k \quad b = \phi(pq)$$

A og b udtrykt med kendte værdier:

$$a = 7 \quad b = 337.744$$

Udgangspunktet:

$$a \cdot u - b \cdot v = 1$$

Indsatte værdier i sætningen:

$$7 \cdot u - 337.744 \cdot v = 1$$

Isoleres for a:

$$7 = 337.744 \cdot 0 + 7$$

Isoleres for b:

$$337.744 = 7 \cdot 48.249 + 1$$

Udregning 2

Jeg har nu fundet resten 1, og kan regne baglæns:

Isoleres for resten 1:

$$1 = 337.744 - 7 \cdot 48.249$$

Erstatter 7:

$$1 = 337.744 - (7 - 337.744 \cdot 0) \cdot 48.249$$

$$1 = 337.744 - 7 \cdot 48.249 + 337.744 \cdot 0 \cdot 48.249$$

Her reducerer jeg udtrykket:

$$1 = 7 \cdot (-48.249) - 337.744$$

Her kan jeg se at værdien af u er negativ. Når den er det, kan jeg ikke regne med den, men jeg kan bestemme en anden værdi der også passer. I den generelle formel, kan jeg nemlig lægge $k \cdot \phi(pq)$ til og trække det samme fra, altså er det lovligt da de ville kunne gå ud med hinanden. Den generelle formel ser ud som vi kender:

$$a \cdot u - b \cdot v = 1$$

Men hvis man erstatter a og b med de værdier som de er lig med, se start af regnestykke, ser den sådan ud:

$$k \cdot u - \phi(pq) \cdot v = 1$$

Her lægger jeg som sagt til og trækker fra:

$$k \cdot u - \phi(pq) \cdot v + k \cdot \phi(pq) - k \cdot \phi(pq) = 1$$

Her samles udtrykket blot

$$k \cdot (u + \phi(pq)) - \phi(pq) \cdot (v + k) = 1$$

Udregning 3

Og her har jeg nu et nyt udtryk for u og v . Dog skal man bare huske at hvis bare ét af resultaterne er negative, skal man lægge til på både u og v , og ikke kun det negative tal. Grunden er selvfølgelig at udtrykket foroven er en ligning og hvis man kun ændrer på ét tal vil ligningen ikke passe.

Nu kan jeg regne med u og v som jeg fik før. Jeg aflæser at $u = -48249$ $v = 1$. Der står godt nok ikke $337.744 \cdot 1$, men da $337.744 = 337.744 \cdot 1$ aflæses v til at være 1. Jeg kan nu beregne de værdier jeg skal bruge:

$$u = -48.249 + 337.744 = 289.495$$

$$v = -1 + 7 = 6$$

Bare for god ordens skyld, tjekker jeg om ligningen passer:

$$7 \cdot 289.495 - 337.744 \cdot 6 = 1$$

Og her har jeg altså min bekræftelse på at tallene passer.

Nu har jeg min værdi af u , og jeg kan forsætte til at bestemme den dekrypterede tekst. Jeg opstiller det relativt simple regnestykke:

$$d = c^u \% n$$

$$d = 140.307^{289.495} \% 338.971$$

Når man vil regne dette ud på en normal regnemaskine, er det slet ikke unormalt at der er sat en grænse for hvor store tal den vil lave med en potens, derfor kan man opleve problemer med at udregne dette "simple" regnestykke. Heldigvis findes der specielle regnemaskiner⁴ som kan udregne disse,

⁴ Defuse Calculator (Online), <https://defuse.ca/big-number-calculator.htm>, besøgt 8/12 2020

men dette problem er noget jeg er nødt til at tage højde for i mit program. Og med lidt hjælp fra regnemaskinen jeg refererer til, kan jeg bestemme at:

$$d = 140.307^{289.495} \% 338.971 = 80 \quad \text{Udregning 4}$$

Og hermed kan jeg bekræfte at algoritmen virker, da jeg satte $m = 80$ og $d = 80$.

Nu ved jeg hvordan jeg skal lave selve algoritmen i mit program, men først skal jeg også overveje hvordan jeg vil udforme mit program, og hvilke krav det skal leve op til.

Programmet

Før jeg overhovedet begynder at overveje at programmere, skal jeg opsætte nogle krav til hvad jeg forventer af mit program.

Krav

Must have

1. Skal kunne kryptere en selvvalgt tekst (tal) til en ciffertekst
2. Skal kunne dekryptere

Could have

3. Skal kunne håndtere "store" tal
4. Skal kunne ændre værdier for kryptering (p, q, k)
5. Automatisk konvertere input til tal

Vil vente

6. Interface

Uddybning af krav:

1. Ved første krav, mener jeg at mit program skal kunne kryptere en selvvalgt tekst, altså et input fra brugeren. Når jeg skriver tekst, mener jeg i virkeligheden tal, da det ikke er et must at kunne kryptere fra tekst til tal.
2. Ved krav nummer to, mener jeg at programmet skal kunne finde frem til det input brugeren har tastet ind. Mine tanker er at generelt skal algoritmen udarbejdes ud fra de beregninger jeg har lavet tidligere, og deraf skal $m = d$.
3. Ved dette "krav", som ikke er et must, mener jeg at programmet skal kunne håndtere tal der er mere end 2, og måske også 3 cifre lange.
4. En anden ting jeg ville synes var en god feature, er at man nemt kan ændre p, q og k værdier. Grunden er selvfølgelig at det ville være relevant hvis man har flere "samtaler" i gang, så ville det give mening at kunne skifte værdier således at forskellige samtaler bruger forskellige tal.
5. En god funktion som kunne være god at have med, er at man i stedet for at indtaste tal som jeg skrev tidligere, kunne man indtaste en brødtekst el. og så have en funktion der kunne lave disse om til tal vha. ACSII eller en anden funktion.
6. Noget jeg ikke vil fokusere på, er interface. Programmet bliver kørt i min editor, og det er ikke et mål at lave en .exe fil som ser "lækker" ud.

Omskrivning, fra matematik til algoritme

Når nu jeg skal skrive min kode, er det vigtigt at jeg kan omskrive den matematik som man laver i hånden, til noget som et program og en computer kan gøre. Jeg tror umiddelbart at jeg har to ting jeg skal fokusere på, fordi de kan være lidt tricky at få til at virke.

Euklids algoritme

Jeg har gennemgået selve algoritmen og tankegangen bag, så dette fokuserer jeg ikke på. Det jeg vil gøre nu, er at jeg vil opstille regnestykket algebraisk. Jeg vil derefter gennemføre beregningen og så bør jeg kunne finde udtrykket der udgør u og v. Når man har sine værdier a og b, siger algoritmen:

$$a = b \cdot D1 + R1$$

Da jeg udelukkende vil bruge bogstaver, bruger jeg D1 for divisor i første omgang, og tilsvarende R1 for resten.

$$b = R1 \cdot D2 + R2$$

Ligesom normalt, flytter jeg b på venstre side, og bruger den foregående rest som nævner for $\frac{b}{R1}$

$$R1 = R2 \cdot D3 + R3$$

I denne udregning antager jeg at $R3 = 1$:

Først isolerer jeg R3, altså 1 $1 = R1 - R2 \cdot D3$

Indsætter udtryk for R2 $1 = R1 - (b - R1 \cdot D2) \cdot D3$

Ganger ind i parentes $1 = R1 - b \cdot D3 + R1 \cdot D2 \cdot D3$

R1 sættes udenfor en parentes $1 = R1(1 + D2 \cdot D3) - b \cdot D3$

Udtryk for R1 indsættes $1 = (a - b \cdot D1) \cdot (1 + D2 \cdot D3) - b \cdot D3$

Ganger parenteser ud $1 = a + a \cdot D2 \cdot D3 - b \cdot D1 - b \cdot D1 \cdot D2 \cdot D3 - b \cdot D3$

A og b sættes udenfor parenteser

$$1 = a(1 + D2 \cdot D3) - b(D1 + D1 \cdot D2 \cdot D3 + D3) \quad \text{Udregning 5}$$

Og ved at aflæse på samme måde som jeg gjorde i afsnittet om Euklids algoritme, har jeg altså mine algebraiske udtryk for u og v når det tager 3 omgange at få resten 1 med Euklids algoritme.

$$\begin{aligned} u &= 1 + D2 \cdot D3 \\ v &= D1 + D1 \cdot D2 \cdot D3 + D3 \end{aligned} \quad \text{Sætning 3}$$

Jeg skal netop bruge disse udtryk i min algoritme, men som man kan se i udregning 2 kan man komme i situationer hvor man finder resten efter kun 2 omgange. Jeg skal altså også finde udtrykkene for disse. Jeg vil bruge samme metode, men blot stoppe tidligere end jeg gjorde i udregning 5.

Ligesom før starter jeg altså med:

$$a = b \cdot D1 + R1$$

$$b = R1 \cdot D2 + R2$$

Og nu antager jeg altså at $R2 = 1$, og jeg skal isolere for den:

$$1 = b - R1 \cdot D2$$

Jeg indsætter udtryk for $R1$

$$1 = b - (a - b \cdot D1) \cdot D2$$

Her ganger jeg ind i parentesen

$$1 = b - a \cdot D2 - b \cdot D1 \cdot D2$$

Her samler jeg udtrykket

$$1 = -a \cdot D2 + b - b \cdot D1 \cdot D2$$

Og her sættes b ud for parentesen $1 = a \cdot -D2 - b \cdot -(1 - D1 \cdot D2)$

Og igen, ligesom før, kan jeg nu blot aflæse udtrykkene for u og v .

$$u = -D2$$

$$v = -(1 - D1 \cdot D2)$$

Sætning 4

Jeg har nu alle de udtryk jeg skal bruge, men jeg skal også kende værdierne af divisorerne og resterne. Det vil jeg blot gøre ved at følge tankegangen bag, så hvis vi går et skridt tilbage til selve algoritmen; Eksemplet viser fremgangsmåden med de viste værdier.

$$a = 61 \quad b = 27:$$

$$61 = 27 \cdot 2 + 7$$

$$27 = 7 \cdot 3 + 6$$

$$7 = 6 \cdot 1 + 1$$

Nu skal jeg da bestemme værdierne for divisorerne og resterne, og hvis jeg starter ved første omgang hvor jeg vil starte med at finde divisoren, vil jeg have den absolutte værdi for:

$$\frac{61}{27}$$

Hvilket jo giver 2. Det vigtige er dog at for at finde $D1$ siger man:

$$D1 = \text{abs}\left(\frac{a}{b}\right)$$

”Abs” betyder at jeg tager den absolutte værdi, altså lad os sige at divisionsstykket gav 2,6 skal man ikke runde op til 3, da resten ville være negativ. Abs sørger altså for at der bliver rundet ned.

Når jeg skal finde resten siger man matematisk jo modulo funktionen med a og b :

$$R1 = a \% b$$

Når man gik til en ny omgang, skiftede variablerne ligesom plads, og det samme sker her. Den tidligere nævner bliver til tæller, og den tidligere rest bliver nævner, for at opstille matematikken som en brøk. Udtrykkene for divisorerne og resterne må være de samme, blot med andre variabler, derfor må anden omgang være:

$$D2 = \text{abs}\left(\frac{b}{R1}\right)$$

$$R2 = b \% R1$$

Og nu kommer det nok ikke som en overraskelse at formlerne for den tredje omgang ser ud som følgende:

$$D3 = \text{abs}\left(\frac{R1}{R2}\right)$$

$$R3 = R1 \% R2$$

Udregning 6

Nu har jeg altså umiddelbart alle de formler jeg skal bruge for at lave mit program.

Faktorisering

Som nævnt i afsnittet med eksemplet, hvor et 6-cifret tal skulle opløftes i et andet 6-cifret tal, vil man typisk støde ind i problemer når man gør dette. Årsagen er egentlig meget simpel, at de programmer som ikke er udviklede til specifikt at kunne regne med store tal har en forholdsvis lav grænse. Det samme gælder hvis man skrev det samme regnestykke ind i sin editor, der er altså formentlig en standart compiler el. som sætter en grænse for hvor store tal man må bruge. Dette gælder selvfølgelig stadig når man vælger den korrekte datatype.

En måde jeg kan arbejde udenom dette problem, er ved at benytte nogle af de matematiske udtryk til min fordel.

I bund og grund er problemet at potensen er for stor. Jeg kan vælge at dele denne op, og selvom man skulle til at lave mange beregninger er denne slags opgaver hvor man laver det samme om og om igen, en af de ting computere er rigtig gode til.

Lad os tage potensen fra eksemplet på side 12:

$$d = 140.307^{289.495} \% 338.971$$

Her er potensen 289.495. Jeg vil lave en simpel faktorisering hvor jeg maks opløfter i 2. Udover det skal jeg også tjekke om potensen er et lige eller ulige tal, og afhængig af svaret skal jeg opløfte i 1. Når jeg skal lave denne faktorisering regner jeg med restklasser, og derfor skal jeg også være lidt opmærksom på hvornår jeg tager modulo funktionen til delresultaterne. For netop et delresultat skal jeg bruge undervejs.

Jeg vil starte med at lave en funktion der tjekker for om potensen er ulige:

$$u \% 2$$

Udregning 7

Hvis denne giver 1, ved jeg at u er ulige og så skal jeg finde delresultatet af:

$$c^1 \% n$$

Udregning 8

Og derefter trække 1 fra potensen u. Når jeg har gjort det, skal jeg i et loop finde delresultatet når jeg opløfter i 2.

$$c^2 \% n$$

Udregning 9

Og herefter trække 2 fra potensen u. Al matematikken er klaret, og nu skal jeg bare sørge for at loopet kører så længe u er større eller lig med 2. Nu er jeg faktisk klar til at kode.

Programmet & kode

Koden til mit program vil ligge vedlagt i en .txt fil, og derudover har jeg lavet en video hvor jeg tester programmet således at man ikke behøver at installere editor og lave nye projekter osv.

Nu da jeg er færdig med mit program vil jeg gerne gennemgå nogle stykker kode.

```
int p = 419; //419
int q = 809; //809
int n = p*q;
int k = 7;
Scanner scan = new Scanner(System.in); //Scanner for scanning inputs

System.out.println("Insert your message (m): "); //Message input from user
String message = scan.nextLine(); //Scanning input
int m = Integer.parseInt(message); //Convert from string to integer

long c = fakto(m,k,n);
```

Billede 1 -Kryptering

Her i den første del kode, er alt arbejdet for krypteringen. Når det er sagt så kalder jeg en funktion som jeg viser lidt senere. Som man kan se har jeg defineret mine værdier for p, q, n og k, som man i princippet kan ændre til nogle andre værdier. Herefter har jeg en scanner der scanner for inputs fra brugeren efter jeg har printet "Insert your message (m): ". Her indtaster brugeren altså den tekst som man vil have krypteret. En detalje er at scanneren umiddelbart gemmer som strings, altså ikke et tal som jeg kan regne med. Derfor konverterer jeg fra string til integer. Jeg kan her bruge integer fordi:

1. brugeren bør kun ville kryptere korte beskeder.
2. Programmet vil crashe hvis inputtet er 7 decimaler eller derover. Det er dog et problem man ikke vil løbe ind i når man har højere værdier for p og q.

Og som jeg nævnte tidligere kan integers holde værdier op til lidt over 2 milliarder⁵ så her kan jeg sagtens nøjes med en integer. Til sidst kan man se at jeg kalder funktionen fakto. Jeg forklarer funktionen lige straks, men det funktionen i bund og grund gør, at den laver regnestykket for c, se evt. Formel 1, og Udregning 1 hvor jeg gennemgår et eksempel. Bid mærke i, at jeg her bruger en datatype kaldet long. En long er den datatype i Java der kan indeholde størst heltal, og da jeg aldrig skal arbejde med decimaltal kan jeg sagtens bruge denne.

Herunder ses funktionen "fakto"

⁵ W3Schools.com, https://www.w3schools.com/java/java_data_types.asp, Datatyper se int, besøgt 12/12 2020

```
static long fakto(long c, long u1, int n) { //c = tekst; u1 = potens ; n = produkt af p&q (modulu)
    long rest = 1; //definerer variabelen
    if (u1%2 == 1) { //Se udregning 7
        rest = (long) c%n; //Se udregning 8 (c^1 = c)
        u1 = u1-1;
    }

    while (u1>=2) {
        rest = (long) (rest*((c*c)%n)%n); //Se udregning 9
        u1 = u1-2;
    }
    return rest;
}
```

Billede 2 -Fakto funktion

Som man kan se, refererer jeg til nogle udregninger i denne rapport. Som der står i koden, tjekker jeg først for om potensen er ulige, hvis den er det skal jeg altså finde resultatet for Udregning 8;

$$c^1 \% n = c \% n$$

Herefter er det vigtigt at jeg trækker 1 fra potensen, da mit udgangspunkt ellers ikke stemmer overens. Herefter starter jeg et while loop. Et while loop kører så længe betingelsen er opfyldt, her er betingelsen altså at u er større eller lig med 2. Grunden er jo, som jeg gennemgik i udregning 9, at jeg her tager resultatet for $u^2 \% n$ og medregner i resten.

Noget man skal lægge mærke til, er i mit while loop; her står der:

```
rest = (long) (rest*((c*c)%n)%n);
```

Umiddelbart kan det godt se ret uoverskueligt ud, men i udregning 9 står der:

$$c^2 \% n$$

Og i virkeligheden er det det samme der står her (i det markerede område):

```
rest = (long) (rest*((c*c)%n)%n);
```

Grunden til at jeg har variabelen "rest" i spil, er selvfølgelig når loopet skal køre igen, for så skal jeg gemme resten fra sidste gang loopet kørte. Når jeg så finder resultatet af udregning 9, skal det multipliceres med den foregående rest, men her skal jeg bruge modulus funktionen igen da tallene ellers kan blive for store. Dét kan jeg godt tillade mig, for når jeg regner med restklasser, er det lovligt! For at opsummere, er funktionens formål at gøre det muligt at regne med store potenser da man ellers kunne risikere at få overflow og dermed smide data væk så man ikke får det rigtige resultat.

Nu antager jeg så at jeg har fundet c, altså ciffer teksten, og jeg skal nu til at dekryptere. Som jeg gjorde i udregning 6, skriver jeg altså alle divisorerne og resterne op og laver den egentlig meget simple udregning. Jeg bruger her integers fordi denne datatype kan holde op til ≈ 2 milliarder i heltal, så umiddelbart bør der ikke komme overflows her.

Derudover kan man se at jeg lige definerer variabler for u og v, da jeg i den næste stump kode er et if statement, og da jeg vil bruge variabelen globalt skal jeg definere den globalt. Jeg skal ikke arbejde mere med variablerne her.

```
//dekrypter
int A = k;
int B = (p-1)*(q-1);

int D1 = A/B;
int R1 = A%B;
int D2 = B/R1;
int R2 = B%R1;
int D3 = R1/R2;
int R3 = R1%R2;
long u1 = 1;
long v1 = 1;
```

Billede 3 -Dekryptering

På Billede 4, ses udførslen af den matematik jeg lavede ved sætning 3 og 4. Grunden var at afhængig af antallet af omgange var udtrykkene for u og v forskellige. I virkeligheden er det en meget simpel ting at implementere efter jeg har beregnet værdierne af resterne, da det så bare er et simpelt regnestykke. Nu har jeg altså givet programmet udtrykket for u og v, og jeg kan endelig komme til de afsluttende trin i min dekryptering.

```
if (R2 == 1) { //hvis resten er 1 ef
    u1 = (long) -D2; //I sætning 4 ha
    v1 = (long) -(1-D1*D2);

} else if (R3 == 1){ //3 omgange
    u1 = (long) 1+(D2*D3); //I sætnin
    v1 = (long) D1+D3+D1*D2*D3;
}
else {
    System.out.println("Error 40");
```

Billede 4 -Definition af u og v

På Billede 5, kan man se hvordan jeg først sørger for at jeg ikke begynder at regne med negative tal ved hjælp af et if statement. Her ”spørger” man om enten u eller v er negative, og hvis en af dem er det, skal programmet køre koden, hvis ikke, skal den break ud og køre videre. Herefter bestemmer jeg d, altså den dekrypterede besked igen ved hjælp af min *fakto* funktion, dog med nogle andre værdier. Jeg vil her henvise til forklaringen og til billede 2 men selvfølgelig med de inputs der står på billede 5. Formlen svarer til *Formel 2*, og denne gennemgik jeg i *udregning 4*.

```
if (u1 <1 |v1<1) { //Hvis u eller v e
    u1 = u1+B;
    v1 = v1+A;
}

long d = fakto(c,u1, n); // c^u1 %n
```

Billede 5 -Dekryptering

Til sidst printer jeg blot resultatet til brugeren, som man kan se på billede 6.

Jeg vil nævne at jeg ikke har fokuseret på optimering af mit program, men udelukkende prøvet at lave et program der udfører opgaven og umiddelbart er let at læse og forstå. Den eneste form for optimering jeg har lavet, er under faktoriseringen. Her har jeg lavet funktionen der finder resultatet for $c^2 \% n$ som jeg kører i stedet for bare at køre med $c \% n$ hvilket jeg formentlig også sagtens kunne have gjort. Teoretisk bør den optimering køre selve faktoreriseringsdelen omkring dobbelt så hurtigt, men i virkeligheden gør det ingen forskel.

```
Insert your message (m):
1017
p 419
q 809
n 338971
k 7
c 144863

Dekryptering:
A 7
B 337744
D1 0
R1 7
D2 48249
R2 1
D3 7
R3 0

U 289495
V 6
Dekrypteret: 1017
```

Billede 6 -Udklip af konsol

Test

Når jeg skal teste mit program, er det selvfølgelig imod de krav som jeg selv har sat. Jeg har sat mine krav ind igen, og under hvert krav vil jeg uddybe hvordan mit produkt har klaret sig i forhold til de krav der skulle med og dem der kunne være gode at have med, men ikke var et must.

Selve testen har jeg ikke kunne være ret kreativ med, da jeg blot kører programmet og følger instruktionerne. Dog mener jeg at ud fra de krav der er sat, er mange ting ikke subjektive og derfor kan jeg tillade mig at lave en objektiv test.

Must have

1. Skal kunne kryptere en selvvalgt tekst (tal) til en cifertekst
 - Mit program kan arbejde med en selvvalgt tekst som man som bruger indtaster. Det kan selvfølgelig også kryptere dette indtastede tekst. Jeg mener altså at man godt kan sige at jeg har opfyldt dette mål.
2. Skal kunne dekryptere
 - Mit program kan dekryptere til det samme input som man har tastet ind, og jeg mener at målet er opfyldt.

Could have

3. Skal kunne håndtere "store" tal
 - Da jeg skrev dette mål, havde jeg umiddelbart i tankerne at programmet skulle kunne håndtere tal der var 2-3 cifre lange. Mit program kan lige nu kryptere og dekryptere tal op til 6 cifre lange. Selvom det ikke er imponerende i forhold til den størrelsesorden som RSA bliver brugt i, i virkeligheden synes jeg bestemt det er et "win" at kunne sige at jeg har opfyldt dette mål.
4. Skal kunne ændre værdier for kryptering (p, q, k)
 - Som jeg skrev under selve kravet, skulle man kunne ændre i værdierne uden at skulle lave fundamentale ændringer i koden. Lige nu kan man sagtens ændre værdierne, så længe man husker at der er nogle betingelser som skal overholdes. Lige nu kan man sagtens ændre p og q til at være eks. 11 og 17 uden at ændre noget andet og stadig have et program der virker. Derfor mener jeg også at jeg har opfyldt målet.
5. Automatisk konvertere input til tal
 - Det sidste mål jeg satte, som ikke var et jeg mente jeg skulle nå, var at programmet automatisk kunne konvertere tekst til tal, og derefter regne med disse. Man kan sagtens se at dette mål har jeg simpelt ikke nået. Det ville være en funktion som ville kunne sætte lys over hvordan krypteringen virker i vores egne computere når de laver sikre forbindelser og krypterer indholdet. Det er selvfølgelig ikke en essentiel del af selve krypteringen, men det ville have givet et fint perspektiv.

Diskussion/Vurdering

Mit produkt i form af et program, mener jeg viser en del gode perspektiver på RSA kryptering, og generelt asymmetriske krypteringstyper. Når det er sagt skal jeg ikke bilde nogen ind at dette er sådan al software der laver RSA ser ud. I mit program er der jo uden tvivl nogle begrænsninger som jeg har været nødt til at arbejde rundt om.

I mit program er der begrænsninger som man ikke ville opleve i en "rigtig" krypteringsalgoritme. Lige nu har jeg eks. en begrænsning på beskedlængden. Grunden til det er at n skal være ligeså lang som den højeste besked man kan sende hvor krypteringen sker korrekt. Hvis jeg skulle til at udvide programmet således at det kunne håndtere tekst, ville jeg også skulle arbejde rundt om hvor vidt jeg skulle kryptere et bogstav ad gangen eller om jeg skulle til at arbejde med så store værdier at jeg kunne kryptere hele teksten i en omgang, og det ville også påvirke valget af p og q værdier. Når det er sagt, var målet bestemt ikke at lave et program der kunne implementeres med det samme. Da jeg har lavet en gennemregning med de samme værdier som jeg bruger i mit program er det overskuelige at se om tingene stemmer overens. Min gennemgang af udregninger kan ses ved udregning 4, og hvis jeg kører programmet med $m = 80$, kan man følge alle delresultater igennem og se at de stemmer overens med de værdier jeg har fået i udregning 4.

En fremtid for programmet kunne selvfølgelig være at udvide til at det kunne tage tekst input, og eventuelt begynde at lave optimeringer til programmet.

Jeg vil opsummere hvor sikkerheden ved RSA ligger. Hvis man kender alle de værdier som jeg har regnet med i opgaven, er det ikke svært at dekryptere en besked.

Hvis jeg genindfører afsender og modtager, og jeg først er afsender; her kender jeg tallet n og k da det er det jeg skal bruge til at kryptere jf. formel 1.

Nu jeg har fået den dekrypterede besked er jeg modtager; her kender jeg p og q . Det er netop med disse at jeg skal bestemme $\phi(pq)$. Sikkerheden ligger altså primært i at det er så svært at primtalsfaktorisere et tal.

Dette er umiddelbart en rigtig fin måde at løse et problem på, dog så er der altid frygten ved hvis nogen finder en måde man kan bestemme disse faktorer hurtigt nok til at det er "farligt" eller i hvert fald sådan at man ikke længere kan kalde forbindelsen sikker længere.

En faktor der altid spiller ind når vi har med teknik og teknologier at gøre, er jo at vi udvikler os og teknologierne udvikler sig så umådelig hurtigt. Nogle kilder siger allerede at man nemt kan primtalsfaktorisere med kvantecomputere⁶, og denne udvikling påvirker i høj grad hvor sikre vores informationer er på nettet. Når kvantecomputerne kommer ud, og hvis man antog at de gennemgik samme udvikling som computere som vi kender, går der måske mindre end 100 år og så har vi kvantecomputere på størrelse med en bærbar. Herefter begynder sikkerhed at være et interessant emne da man så skal finde nye metoder at kryptere på.

Normalt er kryptering ikke set som noget ubrydeligt, men som noget der tager uoverkommelig lang tid, og derfor vil det tage for lang tid og den information man prøver at bryde, vil være værdiløs når

⁶ Aktuell Naturvidenskab, https://aktuelnaturvidenskab.dk/fileadmin/Aktuel_Naturvidenskab/nr-2/AN2-2020-kvantecomputer.pdf, side 19 2. Kolonne, besøgt 15/12 2020

man kan læse den. Udviklingen med computere har fulgt denne fordi krypteringerne bare har skulle være større end hvad computerne kunne regne med. Dog kan der komme helt nye ting til når vi skifter teknologi, altså nye krypteringsmetoder.

Perspektivering

Jeg vil selvfølgelig perspektivere til kryptering som en industri, og alle de programmører der arbejder med kryptering til hverdag. En ting er at vi måske skal skifte metode, men derudover vil man altid prøve at effektivisere krypteringsalgoritmerne. Når de professionelle sidder og koder krypteringsalgoritmer, har de formentlig både særlige sprog og compilers der er optimeret til at kunne lave disse krypteringer. I ”normale” editors er der ofte ikke brug for mange cifre og store regnestykker og de tilhørende compilers er ikke lavet til at kunne bearbejde så meget information på en gang som man beder dem om hvis man prøver at lave et regnestykke med en stor potens. Her vil man altså opleve at få et overflow som jeg lidt har snakket om tidligere. Når man får et overflow er det som ordet antyder, at det flyder over med information, og her kan man altså ikke få fat i alle informationerne, altså tallene som bliver kalkuleret. Det vil altså sige at når man får overflow smides der rigtig meget information væk, som vi så oplever som fejl der ikke giver mening. Dette er uden tvivl noget man i hvert fald har arbejdet på i industrien og man sikkert også optimerer på hele tiden så man som bruger får den bedste oplevelse. Det er ofte fordi man ofte sidder som bruger og venter på et svar når man sender noget ud på nettet, om det så er en anmodning el.

RSA kryptering bruges rigtig ofte og i forskellige sammenhænge. Rigtig mange, næsten alle, SSL certifikater bruger RSA kryptering⁷. Et SSL certifikat er et alle hjemmesider der behandler personlige oplysninger skal have. Det kan være e-forretninger, aviser der bruger log-in systemer ol. I forhold til sikkerhed kan man se at man fra 1. januar 2014 skal bruge en nøglelængde på 2048 bit, hvor man tidligere har brugt nøgler på 1024 bits længde.⁷ På få år har man altså skulle skærpe sikkerheden fordi udviklingen indenfor teknologi går så hurtigt.

Selvom kvatecomputere kan påvirke krypteringens fremtid kan man muligvis stadig bruge RSA på lukkede netværk, eks. I firmaer hvor man sender information rundt lokalt.

Et stort diskussionsemne indenfor kryptologi er sikkerhed i forhold til privatliv. Vores samfund er skredet i samme retning som teknologien. Som computere er blevet mere og mere udbredt er det også klart at trusler, kriminalitet på nettet osv. Skal kunne opspores af politistyrker el. I forhold til kryptering skal man altså også finde et kompromis eller en måde hvorpå de ”gode” kan få adgang til information og alle andre ikke kan. Herunder også omkring overvågning, hvor man kan drage mange etiske problemstillinger ind i billedet. Men idet at nogle, som ikke er modtagere af informationen, skal kunne tilgå den alligevel er et teknisk problem. Enten skal de andre jo have tilgang til nøglerne, eller også skal koden brydes ved brute force, hvor man bare prøver mange kombinationer. Men hvis de ”gode” kan knække koden vha. brute force, så burde ”the bad guys” jo også kunne bruge den eksakt samme metode. Dette er altså et problem, og da de gode mennesker ofte arbejder for staten eller i hemmelighed er det ikke noget man kan finde pålidelige kilder til.

⁷ Xolphin, <https://www.xolphin.dk/support/Terminologi/Algoritmer>, brug af RSA, besøgt 14/12 2020

Konklusion

Kryptering er uden tvivl et emne der vil blive arbejdet meget med for at sikre vores sikkerhed i fremtiden. Den form for kryptering der sker i dag, har vist sig ikke at være et regnestykke som man ellers kunne fristes til at tænke, men nærmere en proces hvor der gemmes en smule information, men kun lige nok til at man med de rigtige tal kan finde frem til den oprindelige besked.

Efter min arbejdsproces kan jeg se at selvom matematikken er ret simpel, man er som programmør nødt til at forudse overflows som fører til fejl fordi information smides væk og disse typer fejl skal bare rettes. Som programmør tror jeg bestemt også at det er interessant at kunne optimere på krypteringsalgoritmer, komme med forskellige løsninger og lave en effektiv kode som gør en forskel hos private personers sikkerhed. RSA kan som sagt også bruges på mindre netværk eller internt i et firma for at holde visse oplysninger private, selvom man jo ikke ville håbe det var nødvendigt.

Min kode i forhold til krypteringskode kan så lidt, og er næsten så ineffektiv som den bliver. Dog så vil jeg igen sige at koden ikke var et forsøg på at lave en kopi der var ligeså god som den der bruges, men blot en kode der skulle kunne vise hvordan man kan lave en simpel version, og påvise et princip om kryptering. Selvom min kode knapt tager et sekund at køre, er dette rigtig lang tid når man sender en forespørgsel. Her skal man også tænke på at de nøgler jeg bruger er 200 gange mindre end dem der bruges i virkeligheden, og derfor er optimering i dette emne utrolig vigtigt for at lave en hurtig og sikker forbindelse.

Teknologien udvikler sig stadig hastigt, og man skifter højst sandsynligt krypteringsmetode når kvatecomputere begynder at blive en del af hverdagen. Udover kvanteteknologi, vil computere generelt true sikkerheden og dette vil forsage en udvikling af kryptering. Ligesom man tidligere har set kan man selvfølgelig hele tiden gøre nøglerne længere og mere uoverskuelige, men selv uden kvanteteknologi kan det være man bliver nødt til at ændre retning i forhold til kryptering fordi det bliver for svært at arbejde med og udvikle på. På sin vis er RSA også en form for traditionel tankegang da man i virkeligheden bruger simpel matematik. Fremtiden kryptering kunne være matematik med et andet perspektiv, eller noget helt andet.

Litteraturliste

Bøger:

Baktoft, Allan, *Matematik I virkeligheden*, Forlag Natskyggen 3. Udgave, side 169-178

Riber, Peter, *Kryptering*, 2. Udgave, side 24-46

Singh, Simon, *Kodebogen (The Code Book) Oversat fra engelsk*, Gyldendal, side 377-379

Net:

Kvantecomputere, Aktuell Naturvidenskab,
https://aktuelnaturvidenskab.dk/fileadmin/Aktuel_Naturvidenskab/nr-2/AN2-2020-kvantecomputer.pdf, side 19 2. Kolonne, besøgt 15/12 2020

Defuse, <https://defuse.ca/big-number-calculator.htm>, online regnemaskine, besøgt 9/12

Cæcar cipher, Wikipedia -Den frie Encyclopedia <https://en.wikipedia.org>, besøgt 9/12 2020

RSA numbers, Wikipedia -Den frie Encyclopedia <https://en.wikipedia.org>, besøgt 9/12 2020

W3Schools.com, https://www.w3schools.com/java/java_data_types.asp, Datatyper se int, besøgt 12/12 2020

Algoritmer, Xolphin, <https://www.xolphin.dk/support/Terminologi/Algoritmer>, brug af RSA, besøgt 14/12 2020

Videoer:

Kørsel af program, <https://youtu.be/Wd70H6B22aA>, Youtube

Bilag

Program (se evt. vedlagt .txt fil)

```
1 package cryptography;
2 import java.util.Scanner;
3
4 public class program {
5
6     public static void main(String[] args) {
7         //krypter
8         int p = 419; //419 11
9         int q = 809; //809 17
10        int n = p*q;
11        int k = 7;
12        Scanner scan = new Scanner(System.in); //Scanner for scanning inputs
13
14        System.out.println("Insert your message (m): "); //Message input from user
15        String message = scan.nextLine(); //Scanning input
16        int m = Integer.parseInt(message); //Convert from string to integer
17
18        long c = fakto(m,k,n); //m^k %n Se Formel 1
19
20
21
22        System.out.println("p "+p+ " \nq "+q+"\nn "+n+"\nk "+k+"\nc "+c);
23
24        //dekrypter
25        int A = k;
26        int B = (p-1)*(q-1);
27
28        int D1 = A/B; //Udregninger på divisorer og rester ses i udregning 6
29        int R1 = A%B;
30        int D2 = B/R1;
31        int R2 = B%R1;
32        int D3 = R1/R2;
33        int R3 = R1%R2;
34        long u1 = 1;
35        long v1 = 1;
36
37
38        if (R2 == 1) { //hvis resten er 1 efter kun 2 omgange
39            u1 = (long) -D2; //I sætning 4 har jeg vist hvordan jeg kom frem til disse resultat
40            v1 = (long) -(1-D1*D2);
41
42        } else if (R3 == 1){ //3 omgange
43            u1 = (long) 1+(D2*D3); //I sætning 3 har jeg vist hvordan jeg kom frem til disse resultat
44            v1 = (long) D1+D3+D1*D2*D3;
45        }
46        else {
47            System.out.println("Error 40");
48        }
49
50
51        if (u1 < 1 | v1 < 1) { //Hvis u eller v er negative, skal jeg addere henholdsvis b og a
52            u1 = u1+B;
53            v1 = v1+A;
54        }
55
56        long d = fakto(c,u1, n); // c^u1 %n Se Formel 2
57
58        System.out.println("\nDekryptering:\nA " + A + " \nB " +B+"\nD1 "+D1+"\nR1 "+R1+"\nD2 "+D2+"\nR2 "+R2+"\nD3 "+D3+"\nR3 "+R3);
59        System.out.print("\nU "+u1+"\nV "+v1+"\nDekrypteret: "+ d);
60    }
61
62    static long fakto(long c, long u1, int n) { //c = tekst; u1 = potens ; n = produkt af p&q (modulu)
63        long rest = 1; //definerer variabelen
64        if (u1%2 == 1) { //Se udregning 7
65            rest = (long) c%n; //Se udregning 8 (c^1 = c)
66            u1 = u1-1;
67        }
68
69        while (u1>=2) {
70            rest = (long) (rest*((c*c)%n)%n); //Se udregning 9
71            u1 = u1-2;
72        }
73        return rest;
74    }
75 }
```